

Question 7

Not yet answered

Marked out of

1.60



To design a counter that counts the even numbers from 0 to 20, how many D-flip flops we will need ?

Select one:

☐ a. 3

☐ b. 4

☐ c. 5

☐ d. 6

Question 8

Answer saved

Marked out of

1.50



The ASSERT statement is used to check output values against expected values. select the Severity options of this command:

Select one or more:

☐ a. Note

☐ a. 3

☐ b. 4

☒ c. 5

☐ d. 6

8

ved
ut of

The ASSERT statement is used to check output values against expected command:

Select one or more:

☐ a. Note

☒ b. failure

☐ c. Wrong

☐ d. warning

☒ e. Error

☐ f. Correct

Previous page

🔍 Type here to search



The truth table below represents a FSM with a 1-bit input (x) and a 1-bit output. Suppose the then processes the input values: 1,0,0,1,1,1 (1 input bit after each Clk). What are state transi

Present State		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

Select one:

- ☐ a. 10, 00, 00, 01, 11, 10
- ☐ b. 10, 00, 10, 01, 00, 10
- ☐ c. 10, 11, 10, 00, 10, 00
- ☐ d. 10, 11, 10, 00, 00, 01

$Q \leq N;$

end if;

end if;

end process X;

end arch;

Select one:

- ☐ a. 4 D latch with enable
- ☐ b. 4-bit register with Asynchronous clear and enable
- ☐ c. 4-bit register with Synchronous reset and enable
- ☐ d. 4-bit register with Asynchronous clear
- ☐ e. 4-bit register with Synchronous reset
- ☐ f. 4-bit register without clear, reset and enable

For each component in top-level entity:

Select one or more:

- ☐ a. When instantiated, mapping its ports is optional.
- ☐ b. When instantiated, it should have unique label
- ☐ c. It can be instantiated multiple times.
- ☐ d. It can be declared multiple times

The following VHDL statements deduce a flip-flop :

Select one or more:

- ☒ a. if (clk'event and clk='1') then q<= d;
- ☒ b. if (rising_edge(clk)) then q<= d;
- ☐ c. if (clk='1') then q<= d;
- ☐ d. q<= d when (clk='1') else q<= 0;

C when "10".

D when others.

The when others covers and catches the following conditions:

Select one:

- ☐ a. when "1"
- ☐ b. when "101"
- ☐ c. when "11"
- ☒ d. when "00"

[Clear my choice](#)

Question 18

Not yet
answered

Marked out of
0.50

 Flag question

The initial value that is assigned to Generic in component could be overwritten in the top-level entity:

Select one:

- ☐ True
- ☐ False

19

20

21

22

25

Finish attempt

Time left 1:20:26

[Previous page](#)

[Next page](#)

- ☒ e. 4 D latch with enable
- ☐ f. 4-bit register without clear, reset and enable

[Clear my choice](#)

Question 10

Not yet
answered

Marked out of
0.60

Flag question

For each component in top-level entity:

Select one or more:

- ☐ a. When instantiated, it should have unique label
- ☐ b. When instantiated, mapping its ports is optional.
- ☐ c. It can be instantiated multiple times.
- ☐ d. It can be declared multiple times

[Previous page](#)



Determine which lines have syntax errors in the accompanying VHDL code:
(Note selecting wrong answers would cause a penalty)

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  entity Error_finder is
4      generic
5          (NUM_STAGES : natural := 16);
6      port
7          ( clk, enable, reset, sr_in: std_logic;
8            sr_out  : out std_logic);
9  end entity;
10 architecture rtl of Error_finder is
11     type sr_length is array ((NUM_STAGES-1) to 0) of std_logic;
12     signal sr: sr_length;
13 begin
14     process (clk, reset)
15     begin
16         if (reset = '1') then
17             sr <= (others=>'0');
18         elsif (rising_edge(clk)) then
19             if (enable = '1') then
20                 sr((NUM_STAGES-1) downto 1) <= sr((NUM_STAGES-2)
21                 sr(0) <= sr_in;
22             end if;
23         end process;
24         sr_out <= sr(NUM_STAGES-1 downto 0);
25     end rtl;

```

Select one or more:

- ☐ a. Line 22
- ☐ b. Line 10
- ☐ c. Line 24
- ☐ d. line 7
- ☐ e. Line 13
- ☐ f. Line 11

- ☐ a. 4-bit register with Asynchronous clear and enable
- ☐ b. 4-bit register with Synchronous reset and enable
- ☐ c. 4-bit register with Synchronous reset
- ☒ d. 4-bit register with Asynchronous clear
- ☐ e. 4 D latch with enable
- ☐ f. 4-bit register without clear, reset and enable

[Clear my choice](#)

Question 10

Not yet
answered

Marked out of
0.60

 Flag question

For each component in top-level entity:

Select one or more:

- ☐ a. When instantiated, it should have unique label
- ☐ b. When instantiated, mapping its ports is optional.
- ☐ c. It can be instantiated multiple times.
- ☐ d. It can be declared multiple times

[Previous page](#)



Lenovo

Select one:

- ☐ a. 4-bit register with Asynchronous clear and enable
- ☐ b. 4-bit register with Synchronous reset and enable
- ☐ c. 4-bit register with Synchronous reset
- ☒ d. 4-bit register with Asynchronous clear
- ☐ e. 4 D latch with enable
- ☐ f. 4-bit register without clear, reset and enable

[Clear my choice](#)

Question 10

Not yet
answered

Marked out of
0.60

[Flag question](#)

For each component in top-level entity:

Select one or more:

- ☒ a. When instantiated, it should have unique label
- ☐ b. When instantiated, mapping its ports is optional.
- ☒ c. It can be instantiated multiple times.
- ☐ d. It can be declared multiple times

[Previous page](#)



Lenovo

10 00 00 01 11 10

T_1 1/11

T_2 1/00

T_3 0101

T_4 1101

T_5 0110

T_6 1/01

T_7 0/10

Select one:

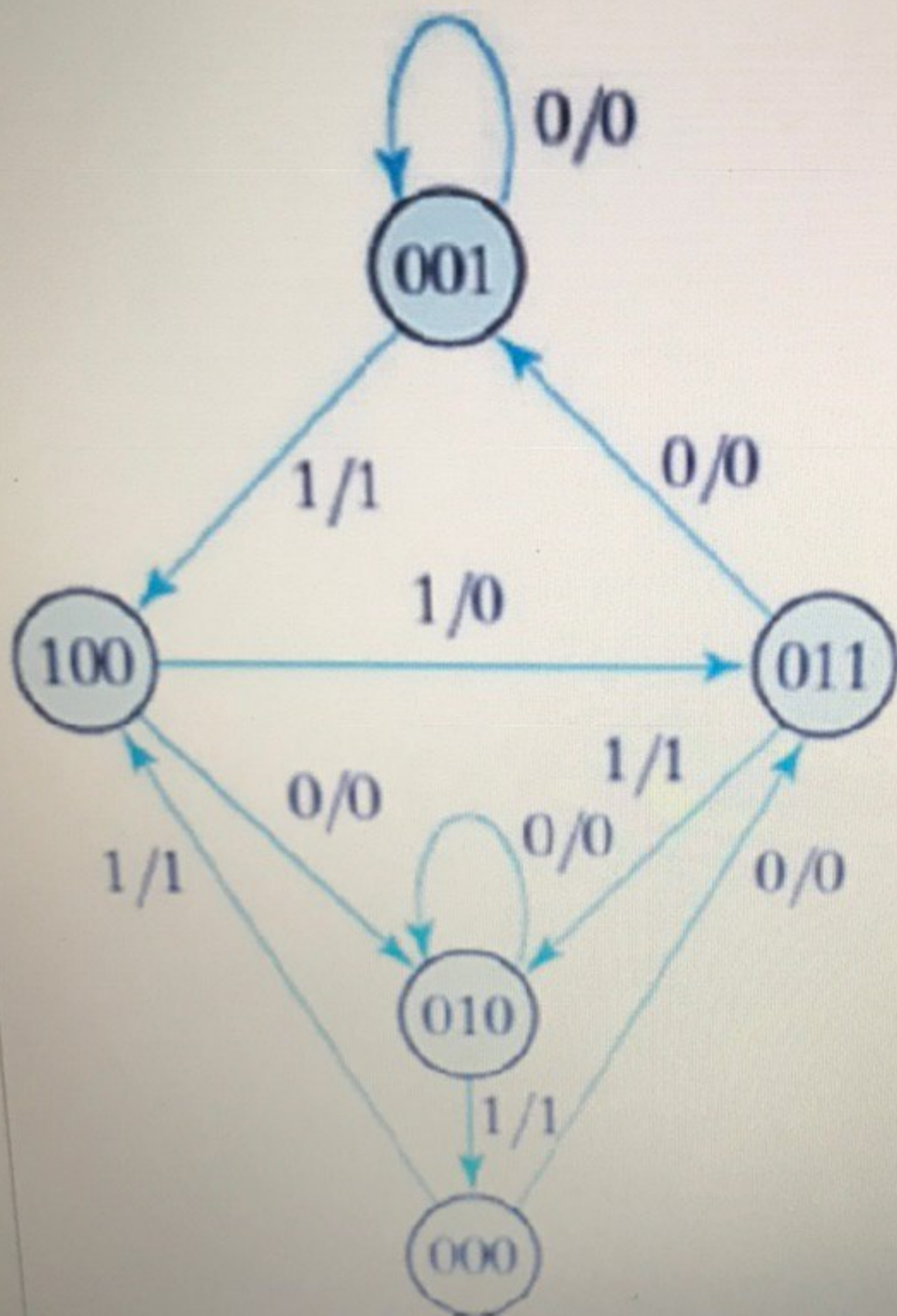
- ☐ a. 10, 00, 10, 01, 00, 10
- ☐ b. 10, 11, 10, 00, 00, 01
- ☐ c. 10, 11, 10, 00, 10, 00
- ☒ d. 10, 00, 00, 01, 11, 10

[Clear my choice](#)



[Previous page](#)

state diagram?



IDENTITY: 0000

lenovo

Question 17

Answer saved

Marked out of

Flag question

In the FPGA design flow, Timing based simulation occurs before synthesis.

Select one:

☒ True

☐ False

Question 18

Answer saved

Marked out of

50

Flag question

Initial values for signals in VHDL are:

Select one or more:

☒ a. Is to be avoided in synthesizable code

☐ b. Are useful in Testbench

☐ c. Is to be avoided in non-synthesizable code

☐ d. Are to be avoided in Testbench

[Previous page](#)

Type here to search





Question 25

Not yet
answeredMarked out of
0.50

Flag question

In the FPGA design flow, Timing based simulation occurs before synthesis

Select one:

☐ True☐ False

Previous page

Finish attempt ...


```
Clock_Loop: Loop  
Clk<= '0';  
wait Clock_Cycle/2;  
Clk<= '1';  
wait Clock_Cycle/2;  
end loop
```

END PROCESS;
END part4_arch;

Select one:

- ☐ a. 0000 0000
- ☐ b. 1011 0000
- ☐ c. 0010 1100
- ☐ d. 0010 0011
- ☒ e. 0011 0000
- ☐ f. 0000 1100

[Clear my choice](#)

[Previous page](#)

Question 21

Not yet
answered

Marked out of
0.60

Flag question

For each component in top-level entity:

Select one or more:

- ☐ a. When instantiated, it should have unique label
- ☐ b. It can be instantiated multiple times.
- ☐ c. When instantiated, mapping its ports is optional.
- ☐ d. It can be declared multiple times

Question 22

Answer saved

Marked out of
0.50

Flag question

The following VHDL statements deduce a flip-flop :

Select one or more:

- ☐ a. $q \leq d$ when $(clk='1')$ else $q \leq 0;$
- ☒ b. if $(clk'event \text{ and } clk='1')$ then $q \leq d;$

Question 4

Complete

er

ed out of

tag question

To test a 2x4 Decoder using Records, complete the following record declaration and

type test_vector is record

d : std_logic_vector(2 downto 0);

o : std_logic_vector (4 downto 0);

end record;

type test_vector_array is array (0 to 2) of test_vector;

constant test_vectors : test_vector_array := (

('0' , , , , ,),

(, , , , ,),

(, , , , ,),

(, , , , ,));

Please put an answer in each box.

Type here to search



end if;

when C=>

if input = '1' then

output <= '1' ;

else

output <= '0' ;

end if;

when D=>

if input = '1' then

output <= '1' ;

else

output <= '0' ;

end if;

when E =>

if input = '1' then

output <= '0' ;

else

output <= '0' ;

end if;

end case;

end process;

end rtl;

Type here to search



output <= '1'

else

output <= '0'

end if;

when B=>

if input = '0' then

output <= '0'

else

output <= '1'

end if;

when C=>

if input = '1' then

output <= '1'

else

output <= '0'

end if;

when D=>

if input = '1' then

output <= '1'

else

output <= '0'

Type here to search



if input = '1' then

Next_state <= E

else

Next_state <= D

end if;

when B =>

if input = '1' then

state <= E

else

state <= D

end if;

when C =>

if input = '1' then

state <= A

else

state <= C

end if;

when D =>

if input = '1' then

state <= C

else

state <= B

Type here to search



when E=>

if input = '1' then

state <= D ;

else

state <= C ;

end if;

end case;

end process;

process (input, state)

begin

case state is

when A=>

if input = '1' then

output <= '1' ;

else

output <= '0' ;

end if;

when B=>

if input = '0' then

output <= '0' ;

else

Type here to search



END COMPONENT;

SIGNAL A, B : STD_LOGIC_VECTOR(7 DOWNTO 0);

SIGNAL S : STD_LOGIC_VECTOR(8 DOWNTO 0);

SIGNAL carry : STD_LOGIC;

BEGIN

-- regn (R, Clock, Resetn, Q)

A_reg: regn PORT MAP (SW, KEY(1), KEY(0), A);

B <= SW;

-- drive the displays through 7-seg decoders

digit_3: hex7seg PORT MAP (A, , HEX3);

digit_2: hex7seg PORT MAP (A, , HEX2);

digit_1: hex7seg PORT MAP (B, , HEX1);

digit_0: hex7seg PORT MAP (B, , HEX0);

S <= ;

carry <= S(8);

digit_5: hex7seg PORT MAP (S, , HEX5);

digit_4: hex7seg PORT MAP (S, , HEX4);

LEDR(0) <= carry;

Type here to search



END COMPONENT;

COMPONENT regn

PORT (R : IN STD_LOGIC_VECTOR(7 DOWNTO 0);

Clock, Resetn : IN STD_LOGIC;

Q : OUT STD_LOGIC_VECTOR(7 DOWNTO 0));

END COMPONENT;

SIGNAL A, B : STD_LOGIC_VECTOR(7 DOWNTO 0);

SIGNAL S : STD_LOGIC_VECTOR(8 DOWNTO 0);

SIGNAL carry : STD_LOGIC;

BEGIN

— regn (R, Clock, Resetn, Q)

A_reg: regn PORT MAP (SW, KEY(1), KEY(0), A);

B <= SW;

— drive the displays through 7-seg decoders

digit_3: hex7seg PORT MAP (A (7 DOWNTO 4) ⇅ , HEX3);

digit_2: hex7seg PORT MAP (A (3 DOWNTO 0) ⇅ , HEX2);

digit_1: hex7seg PORT MAP (B (7 DOWNTO 4) ⇅ , HEX1);

digit_0: hex7seg PORT MAP (B (3 DOWNTO 0) ⇅ , HEX0);

S <= ('0' & A) + ('0' & B) ⇅ ;

carry <= S(8);

digit_5: hex7seg PORT MAP (S (7 DOWNTO 4) ⇅ , HEX5);

digit_4: hex7seg PORT MAP (S (3 DOWNTO 0) ⇅ , HEX4);

LEDR(0) <= carry;

Question 25

Answer saved

Marked out of
3.00

Flag question

To test a 2x4 Decoder using Records, complete the following record declaration and its constant of array:

type test_vector is record

d : std_logic_vector(2 downto 0);

o : std_logic_vector (2 downto 0);

end record;

type test_vector_array is array (0 to 5) of test_vector;

constant test_vectors : test_vector_array := (

('0' , '0' , '0' , '0' , '0' , '0'),

('0' , '1' , '1' , '0' , '1' , '0'),

('1' , '0' , '1' , '0' , '0' , '1'),

('1' , '1' , '0' , '1' , '1' , '1'));

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25			

Finish attempt ...

Time left 0:31:36

[Previous page](#)
[Finish attempt](#)